Assignment 1 - Compilation pipeline and its different components

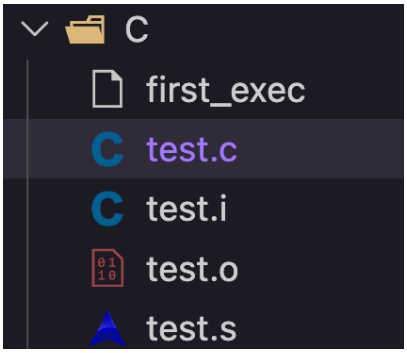
Group Members

- 23074016
- 23074017
- 23074018

Part 1: C

```c
#include <stdio.h>
#define SUCCESS 0

int main (int argc, char **argv) {
  int c;
  //scanf("%d", &c );
  printf("\n HEllo World %d %d\n", c, argc);
  return (SUCCESS);
}
```



1) Preprocessing

`test.c -> test.i`

```
gcc -E test.c -o test.i
```

`-E` : Directs the compiler to stop after the **preprocessing** stage without compiling it into assembly.

2) Target assembly

`test.i -> test.s`

```
gcc -S test.i -o test.s
```

`-S` : Instructs the compiler to stop after the compilation stage and output raw assembly code rather than continuing to the assembler stage

`gcc -S -fverbose-asm test.i -o test.s`
fverbose-asm: adds comments to the generated assembly

3) Relocatable machine code

`test.s -> test.o`

```
gcc -c test.s -o test.o
```

`-c` : Tells GCC to assemble the source file but **stop before the linking stage**. This generates a relocatable object file.

4) Target machine code

`test.o -> first_exec`

```
gcc test.o -o first_exec
```

Runnning the executable

`./first_exec`

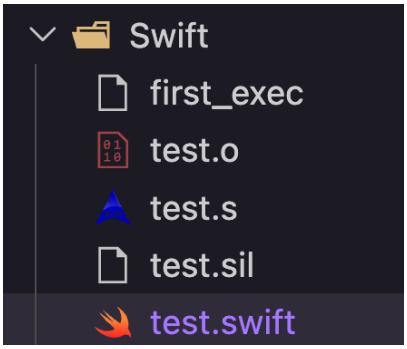
```
compiler/Assignment-1/C via ⬢ v22.12.0 via Ⓒ base
➜ ./first_exec

 HEllo World 1 1
```

Part 2: Swift

```swift
func calculateSum() -> Int {
    var sum = 0
    for number in 1...10 {
        sum += number
    }
    return sum
}

let result = calculateSum()
print("The sum of numbers from 1 to 10 is: \(result)")
```



1. Swift Intermediate Language (SIL)

Instead of preprocessing, Swift first converts the code into SIL, which does Swift-specific analysis (like memory management and type checking).

`test.swift -> test.sil`

Bash

```
swiftc -emit-sil test.swift -o test.sil
```

- `-emit-sil` : Directs the compiler to output the optimized Swift Intermediate Language code and stop. (You can also use `-emit-silgen` for unoptimized, raw SIL

2. Target Assembly

`test.sil -> test.s`

```
swiftc -S test.sil -o test.s
```

- `-S` (or `-emit-assembly`): Instructs the compiler to output raw assembly code rather than continuing to the assembler stage.

3. Relocatable Machine Code

`test.s -> test.o` *(can also pass* `test.swift` *directly)*

```
gcc -c test.s -o test.o
```

- `-c` (or `-emit-object`): Tells `swiftc` to assemble the file and stop before the linking stage, generating a relocatable object file.

NOTE: `swiftc -c test.s -o test.o` does not work because its driver refuses to accept `.s` files as **input**. Once the code is lowered to raw assembly language, `swiftc` essentially considers its specific job done and expects the system's standard assembler to take over.

4. Target Machine Code (Linking)

`test.o -> first_exec`

```
swiftc test.o -o first_exec

OR

gcc test.o -o first_exec
```

Running the executable

```
./first_exec
```

```
compiler/Assignment-1/Swift via ⬢ v22.12.0 via Ⓒ base
➜ ./first_exec
The sum of numbers from 1 to 10 is: 55
```

single command to get executable file

```
swiftc -save-temps -o first_exec test.swift
```